

Diagnostic — Blocage WireGuard sur hotspot mobile

1. Contexte

Test de **wg_auto.sh** en mode déplacement, connexion via hotspot mobile (Port_Armel). Objectif : valider la bascule automatique maison/déplacement de PC1 en conditions réelles.

2. Principe — handshake vs transport de données

Un tunnel WireGuard fonctionne en deux temps distincts, qui peuvent être bloqués indépendamment l'un de l'autre par un réseau intermédiaire :

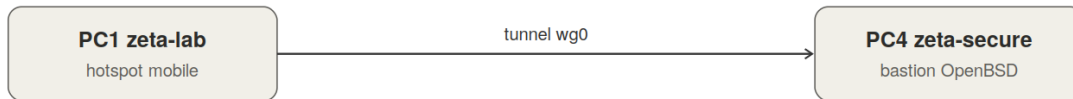
- 1. **Handshake** : poignée de main cryptographique initiale + paquets keepalive périodiques. Paquets très petits et peu fréquents.
- 2. **Transport de données** : trafic applicatif réel (ping, SSH, synchronisation cluster). Paquets plus gros et/ou plus soutenus dans le temps.

Un opérateur mobile ou un pare-feu peut laisser passer le premier tout en filtrant le second (DPI ciblant le trafic soutenu, NAT asymétrique, règle de pare-feu trop restrictive côté serveur). C'est exactement ce qui a été observé ici.

3. Résultats des tests

Étape	Résultat
Connexion hotspot mobile	OK (IPv4 CGNAT + IPv6 disponible)
Tunnel wg0, endpoint IPv4 (DuckDNS)	Handshake : aucune réponse (0 B received)
Endpoint basculé en IPv6	Handshake OK (124 B puis 744 B received, keepalive stable ~3 min)
Ping 10.10.0.1 (endpoint IPv6)	KO — 100% perte, avant et après délai de 150s
SSH TCP port 22 vers 10.10.0.1	KO — timeout total
Test MTU (ping -s 8)	KO aussi — fragmentation écartée comme cause

4. Schéma — déroulé du diagnostic



Test 1 — endpoint IPv4 (DuckDNS)

Handshake : aucune reponse (0 B recus)

Cause : hotspot bloque le trafic WireGuard en IPv4 (CGNAT / DPI)

Test 2 — endpoint IPv6

Handshake OK (keepalive stable, 744 B recus)

Ping 10.10.0.1 : echec (100% perte)

SSH TCP :22 : timeout total

Test MTU (ping -s 8) : echec aussi, fragmentation ecartee comme cause

Handshake OK != transport de donnees fonctionnel



Interference observee

Le dispatcher NetworkManager 99-wg-auto a remonte wg0 seul
(renouvellement DHCP) pendant les verifications manuelles



Conclusion

Le tunnel de controle (handshake) fonctionne en IPv6

Le trafic de donnees ne traverse pas, quelle que soit sa taille

Hypothese : filtrage pf sur PC4, ou DPI mobile plus fin sur le trafic soutenu

A verifier en LAN maison (logs pf de PC4)

5. Interférence observée — dispatcher NetworkManager

Pendant les vérifications manuelles, le dispatcher `/etc/NetworkManager/dispatcher.d/99-wg-auto` s'est redéclenché seul suite à un renouvellement DHCP sur le hotspot, remontant `wg0` automatiquement. Comportement correct du script en soi, mais source de confusion pour du débogage manuel pas-à-pas.

Bonne pratique retenue : désactiver temporairement le dispatcher pendant une session de diagnostic manuel (`sudo chmod -x 99-wg-auto` puis `chmod +x` après).

6. Conclusion

IPv4 : le hotspot bloque le handshake (CGNAT/DPI) — confirmé.

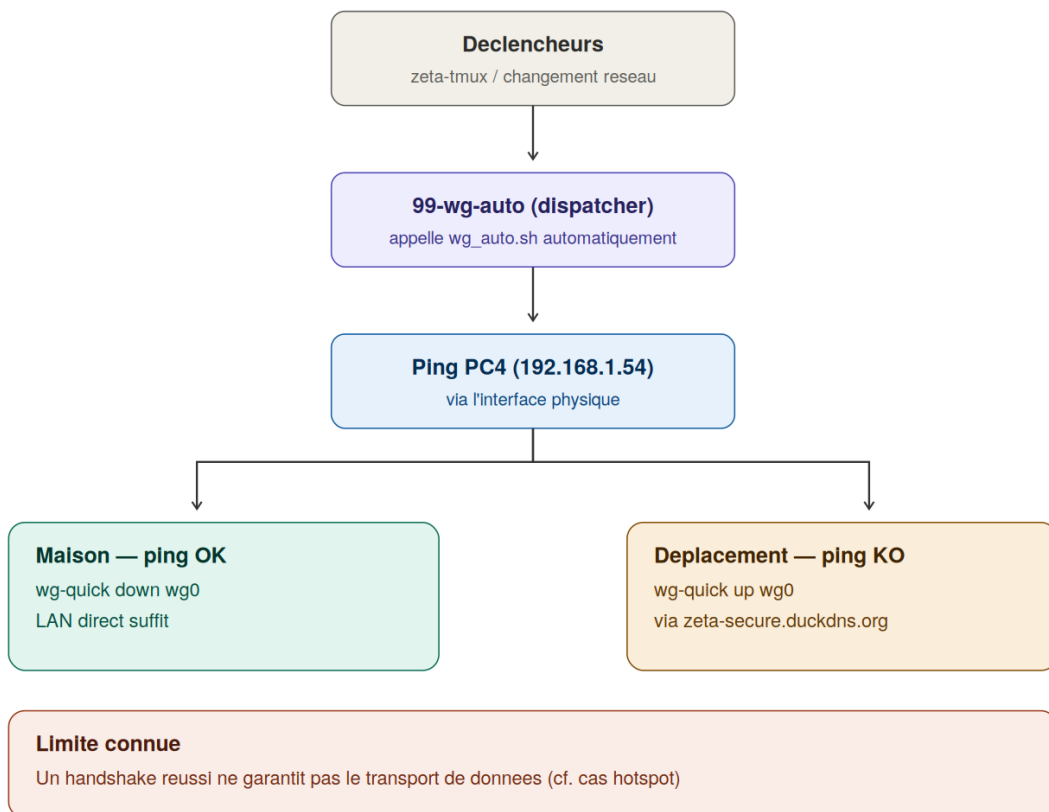
IPv6 : handshake + keepalive OK, stables dans le temps, mais aucune donnée applicative ne traverse (ni ICMP ni TCP), même après attente et à taille de paquet minimale.

Cause probable : filtrage pf sur PC4 (OpenBSD) au niveau du forwarding vers 10.10.0.0/24, ou DPI/NAT plus fin de l'opérateur mobile bloquant spécifiquement le trafic de données soutenu au-delà du handshake.

Non résolu à distance : PC4 n'est joignable que via ce tunnel, donc logs pf inaccessibles depuis le hotspot.

7. Vue globale — automatisation maison / déplacement

Indépendamment de ce blocage réseau ponctuel, l'automatisation de `wg_auto.sh` elle-même fonctionne correctement dans les deux contextes (maison et déplacement), déclenchée soit par le dispatcher réseau, soit par `zeta_tmux.sh` au lancement du cluster.



Point de vigilance : un handshake réussi ne garantit pas que le tunnel transporte des données — c'est une limite réseau externe, pas un défaut du script.

8. Prochaine étape

Vérifier les logs et règles pf sur PC4 en LAN maison (comparer avec §7 du wiki réseau). Endpoint wg0.conf restauré en IPv4 (zeta-secure.duckdns.org:51820) après le test.